Grep is All You Need v1.1 — The LocalKin Team

Grep is All You Need: Zero-Preprocessing Knowledge Retrieval for LLM Agents

The LocalKin Team *Correspondence:* `contact@localkin.ai` *Project:* https://localkin.dev | https://github.com/LocalKinAI

Position Paper — April 2026 (v1.1, updated 2026-04-25) DOI: 10.5281/zenodo.19777260

Abstract

Retrieval-Augmented Generation (RAG) has become the dominant paradigm for grounding Large Language Model (LLM) agents in domain-specific knowledge. The standard approach requires selecting an embedding model, designing a chunking strategy, deploying a vector database, maintaining indexes, and performing approximate nearest neighbor (ANN) search at query time. We argue that for domain-specific knowledge grounding — where the vocabulary is predictable and the corpus is bounded — this entire stack is unnecessary. We present *Knowledge Search*, a two-layer retrieval system composed of (1) `grep` with contextual line windows over raw source texts and (2) `grep` over LLM-compiled per-source concept and FAQ files generated nightly by a free, local, autonomous compilation pipeline. Deployed in production across **76 specialized LLM agents** serving three knowledge domains (Traditional Chinese Medicine, Christian spiritual classics, and U.S. civics), grounded in **~500 primary source texts and ~180 MB of corpus**, our approach achieves 100% retrieval accuracy with sub-10ms latency, zero preprocessing per query, zero additional memory footprint, and zero infrastructure dependencies. We also document a reproducible failure-and-recovery cycle (0/5 fabricated quotes → 4/4 grep-verified quotes after a one-commit fix) that demonstrates the architecture's safety properties are recoverable through prompt hygiene alone — no retraining, no infrastructure change. The key insight is simple: retrieval does not need intelligence. The LLM is the intelligence.

Keywords: retrieval-augmented generation, knowledge grounding, LLM agents, information retrieval, domain-specific AI, zero-hallucination retrieval, autonomous corpus growth

---

1. Introduction

The year is 2026, and every LLM application tutorial begins the same way: choose an embedding model, chunk your documents, spin up a vector database, build an index, and pray that approximate nearest neighbor search returns the right passages. This pipeline — collectively known as Retrieval-Augmented Generation (Lewis et al., 2020) — has become so ubiquitous that it is treated as a law of nature rather than what it actually is: an engineering choice with significant tradeoffs.

We propose an alternative. For domain-specific knowledge grounding, where the source texts are known, the vocabulary is predictable, and the corpus fits within reasonable bounds, the entire RAG stack can be replaced by two Unix utilities that predate the World Wide Web: `grep` and `cat`.

This is not a toy experiment. Our system, *Knowledge Search*, is deployed in production as part of LocalKin, a multi-agent AI platform. As of April 2026, it serves as the knowledge backbone for **39 Traditional Chinese**

Medicine (TCM) agents, **37 Christian spiritual direction agents**, and a U.S. citizenship coaching agent — **76 specialized agents** in total (free-tier whitelist), grounded in approximately **500 primary source texts (~180 MB)** spanning two languages and four-and-a-half millennia of human thought (from the Yellow Emperor's Inner Canon to living National Grand Masters; from Irenaeus, 130 AD, to T. Austin-Sparks, 1971). The system serves all 76 agents from a single Mac mini.

The results are not close. Knowledge Search achieves 100% retrieval accuracy at sub-10ms latency with zero per-query preprocessing, while vector RAG systems typically deliver 85-95% accuracy at 50-200ms latency after hours of upfront preprocessing. We do not claim this approach works for everything. We claim it works remarkably well for the class of problems where most practitioners reflexively reach for vector databases.

Furthermore, the architecture's "zero-hallucination" property is not aspirational — it is reproducible. We document a one-day cycle (Section 6.5) in which a deliberate prompt-engineering regression collapsed citation accuracy to 0/5 grep-verified quotes; auto-stripping 41 fake-quote markers across 79 soul prompts and adding a citation hard-rule restored it to 4/4 — recovered by prompt hygiene alone, with no retraining and no infrastructure change.

This paper is structured as follows. Section 2 examines the hidden costs of the standard RAG pipeline. Section 3 presents our two-layer retrieval architecture. Section 4 describes the knowledge corpus. Section 5 provides comparative analysis. Section 6 explains why this approach works, and includes a reproducibility addendum (§6.5). Section 7 honestly addresses its limitations. Section 8 discusses production integration. Section 9 covers autonomous corpus growth — a daily cron-driven pipeline that has compiled 47→345 concept/FAQ entries in 17 days at zero monetary cost. Section 10 reflects on what this means for the field.

---

2. The Hidden Costs of Vector RAG

The standard RAG pipeline is presented as a solved problem, but each stage introduces compounding complexity, latency, and — most critically — information loss.

2.1 Embedding Model Selection

The first decision is which embedding model to use. OpenAI's `text-embedding-3-large`? Cohere's `embed-v3`? A fine-tuned Sentence-BERT variant? Each model encodes different semantic assumptions. A model trained primarily on English web text will produce poor embeddings for Classical Chinese medical terminology. The choice is consequential, yet there is no principled way to make it without extensive evaluation — evaluation that requires the very retrieval system you have not yet built.

2.2 Chunking Strategy

Documents must be split into chunks before embedding. But how? Fixed-size windows of 512 tokens? Recursive splitting by headers? Semantic chunking based on topic boundaries? Every strategy is a lossy compression of the original text. A passage about the herb 黄芪 (Astragalus root) that spans a chunk boundary will be split into two fragments, neither of which fully captures the original meaning. The chunk size directly determines the ceiling of retrieval quality, yet it must be chosen before any retrieval has occurred.

2.3 Vector Database Operations

The embedded chunks must be stored in a vector database — Pinecone, Weaviate, Chroma, Qdrant, pgvector, or one of the dozens of alternatives that have emerged since 2023. Each requires its own deployment, configuration, and operational expertise. Each has different consistency guarantees, scaling characteristics, and failure modes. For a solo developer or small team, this is not a trivial dependency — it is an entire subsystem that must be monitored, backed up, and maintained.

2.4 Approximate Nearest Neighbor Search

At query time, the user's question is embedded and compared against the stored vectors using approximate nearest neighbor algorithms (HNSW, IVF, or similar). The word "approximate" is doing heavy lifting here. ANN search trades accuracy for speed, and the tradeoff is not always favorable. A query about 伤寒论 (Treatise on Cold Damage) might retrieve passages about 温病 (Warm Disease) because the embeddings are geometrically close — the texts discuss overlapping symptoms. The retrieved passages are plausible but wrong, and the LLM has no way to know this.

2.5 The Maintenance Tax

When new documents are added, the index must be rebuilt. When the embedding model is updated, the entire corpus must be re-embedded. When chunk sizes are adjusted, everything starts over. This maintenance tax is invisible in demos but dominates the operational cost of production systems.

---

3. Our Approach: Two-Layer Knowledge Retrieval

Knowledge Search replaces the entire RAG pipeline with two layers, each implemented as a single system call.

3.1 Layer 1: grep — Exact Contextual Search

The first layer performs keyword search over raw source texts using `grep` with an 8-line context window:

```
grep -r -i -n -C 8 "$query" "$knowledge_dir"
```

The flags are straightforward: - `-r`: recursive search across all files in the knowledge directory - `-i`: case-insensitive matching - `-n`: include line numbers for source attribution - `-C 8`: return 8 lines of context before and after each match

This is not sophisticated. That is the point. When a user asks about 黄芪的功效 (the effects of Astragalus), the search term 黄芪 will appear verbatim in every relevant passage of the TCM corpus. There is no embedding to misinterpret, no chunk boundary to split the answer, no approximate search to return a near-miss. The match is exact, the context is complete, and the retrieval is deterministic.

Multiple matches across different source texts are concatenated and passed to the LLM, which synthesizes the answer. The LLM sees the original text exactly as it was written, with surrounding context intact. No information has been lost.

Performance characteristics: - Latency: 2-8ms for a 162-file corpus on commodity hardware - Accuracy: 100% recall for queries containing domain vocabulary - Preprocessing: none - Memory overhead: none (files remain on disk, read on demand)

3.2 Layer 2: grep — Per-Source LLM-Compiled Concept and FAQ Bridge

Not every query contains a greppable keyword in the language of the source corpus. A Chinese-speaking user might ask 史百克对"破碎"的看法 ("Austin-Sparks's view on brokenness") while the source corpus is entirely in English (`brokenness`, `broken vessel`, `the cross deals with the natural man`). Pure Layer 1 grep over English text would not match Chinese query terms.

Layer 2 solves this with a **per-source concept-and-FAQ compilation layer**, generated automatically by a daily cron job. For every source file in the corpus, the system maintains two small companion files:

```
input/<domain>/<author>/                        # Layer 1: raw originals
├── 01_school_of_christ.txt        (176 KB)
└── _compiled/                                   # Layer 2: auto-generated
    ├── 01_school_of_christ_concepts.md   (~3 KB, key concepts + verbatim quotes)
    └── 01_school_of_christ_faq.md        (~2.5 KB, 5–8 Q&A pairs)
```

The compilation process is run by `knowledge_compile.py`, a 400-line Python skill that:

1. Reads one source file (truncated to 150 KB to fit LLM context)
2. Sends two prompts to a local LLM:
   - **Concepts prompt**: extract 5–10 core concepts as `<name>: definition + 1 key verbatim quote`, plus 3–5 chapter-attributed quotes
   - **FAQ prompt**: generate 5–8 Q&A pairs in the form a student would actually ask
3. Writes outputs to the `_compiled/` sibling directory
4. Skips on subsequent runs if both compiled files already exist (idempotent)

Critical implementation choice (revised 2026-04-24): The primary LLM was migrated from a paid API (Anthropic Haiku 4.5 via OAuth) to a **local Ollama deployment of Kimi 2.6** (`kimi-k2.6:cloud`). The migration reduced per-file compilation cost from ~$0.01 to **$0.00** with no quality regression on Chinese spiritual and TCM content. Haiku is retained as a fallback if the local Kimi instance returns empty or short output. The full call site is:

```python
def call_kimi(prompt):
    payload = json.dumps({
        "model": "kimi-k2.6:cloud",
        "prompt": SYSTEM_PROMPT + "\n\n" + prompt,
        "stream": False,
        "options": {"temperature": 0.3, "num_predict": 4000},
    })
    return curl_post("http://localhost:11434/api/generate", payload)
```

When `knowledge_search` is invoked, **both layers are searched together** with the same `grep` invocation:

```bash
grep -r -i -n -C 8 "$query" "$knowledge_dir"   # walks both raw files and _compiled/
```

This is the architectural inversion from the more conventional "fallback" framing: Layer 2 is **not a degradation path**, it is a **parallel concept-bridge**. When the user query contains a term that appears only in Layer 2 (e.g., a Chinese concept name corresponding to an English source), grep returns hits from the `_concepts.md` or

`_faq.md` files, which the LLM uses to locate the correct passage in the original source. The agent then quotes from Layer 1 with chapter attribution.

Sizing: With 500 source files, the per-source compilation overhead is approximately 5.5 KB × 500 ≈ 2.7 MB of compiled material — a negligible fraction of the 180 MB raw corpus. `grep` performance is unchanged.

Generation cadence: A daily cron task runs `compile_batch` over each domain at 5:00, 5:30, and 6:00 AM (staggered to avoid concurrent Ollama load), processing 30 new source files per domain per night. As of 2026-04-25 the system has compiled 345/505 entries (68%) in 17 days, growing at ~90 entries/night, $0/night.

3.3 The Design Principle

The architecture embodies a single principle: **retrieval does not need intelligence; the LLM is the intelligence.**

Vector RAG systems attempt to build intelligence into the retrieval layer — semantic understanding via embeddings, relevance ranking via similarity scores, re-ranking via cross-encoders. This is engineering effort applied to the wrong layer. The LLM is already the most powerful language understanding system in the pipeline. Give it the raw text and let it do what it does best.

---

4. Knowledge Corpus

Knowledge Search is deployed across three distinct knowledge domains, each with different characteristics. As of 2026-04-25 the production corpus comprises approximately **500 primary source texts totaling ~180 MB**, broken down as follows:

Domain	Source files	Layer 1 size	Layer 2 compiled (Apr 25)
TCM (Chinese)	171	~93 MB	115 / 171 (67%)
Christian spiritual (English)	114	~56 MB	115 / 115 (100%)
Christian spiritual (Chinese)	219	~25 MB	115 / 219 (52%)
TCM (English stubs)	5	<1 MB	5/5
Total	**509**	**~180 MB**	**350 / 510 (69%)**

4.1 Traditional Chinese Medicine (171 ZH source files, 39 master agents)

The TCM corpus comprises classical medical texts spanning from the Yellow Emperor (~2500 BCE) to living National Grand Masters (still practicing as of 2026), organized as a roster of 39 master agents grouped by historical tier:

- **Tier 1 — Classical Sages (16 masters)**: Huang Di (Yellow Emperor), Zhang Zhongjing, Hua Tuo, Huangfu Mi, Sun Simiao, Liu Wansu, Zhang Zihe, Li Dongyuan, Zhu Danxi, Li Shizhen, Zhang Jingyue, Wu Jutong, Ye Tianshi, Wang Qingren, Huang Yuanyu, Fu Qingzhu
- **Tier 2 — Republican Era (5)**: Zhang Xichun, Cao Yingfu, Lu Yuanlei, Pu Fuzhou, Ding Ganren
- **Tier 3 — Modern Classical Formula Revival (4)**: Hu Xishu, Liu Duzhou, Huang Huang, Fan Zhonglin
- **Tier 4 — Contemporary National Grand Masters (10)**: Deng Tietao, Zhu Liangchun, Jiao Shude, Yan Dexin, Zhou Zhongying, Wang Qi, Lu Zhizheng, Ren Jixue, Gan Zuwang, Qiu Peiran

- **Special / contemporary (4)**: Zheng Qinan (Fire Spirit School), Liu Lihong (*Thinking Through Chinese Medicine*), Ni Haixia, Hao Wanshan

Foundational works include 黄帝内经 (*Huangdi Neijing*, ~9 source files), 伤寒论 (*Shanghan Lun*, 18 versions), 本草纲目 (*Bencao Gangmu*, 5.2 MB), 千金方 / 千金翼方 (Sun Simiao, ~11 MB), 四圣心源 (Huang Yuanyu, 10 volumes), and contemporary Renji series transcripts (Ni Haixia, 6 lecture-text files / 5.8 MB).

These texts are written in Classical Chinese with highly standardized medical vocabulary. The term 气虚 (qi deficiency) has meant the same thing for two thousand years. It does not require semantic interpretation — it requires exact retrieval.

4.2 Christian Spiritual Classics (333 source files across EN+ZH, 37 master agents)

The spiritual corpus covers 1,900 years of contemplative and mystical Christian literature, organized as 37 master agents grouped into six tiers:

- **Tier 1 — Church Fathers (4, 130–430 AD)**: Irenaeus, Athanasius, Chrysostom, Augustine
- **Tier 2 — Contemplative Mystics (9, 14c–1897)**: *Cloud of Unknowing* author, Thomas à Kempis, Teresa of Ávila, John of the Cross, Brother Lawrence, Molinos, Francis de Sales, Madame Guyon, Thérèse of Lisieux
- **Tier 3 — Reformation & Puritan (3, 1483–1688)**: Martin Luther, John Calvin, John Bunyan
- **Tier 4 — Great Awakening & Revival (9, 1700–1898)**: Zinzendorf, Jonathan Edwards, John Wesley, George Whitefield, Charles Finney, Charles Spurgeon, D. L. Moody, Andrew Murray, George Müller
- **Tier 5 — Missions & 20th-c. Revival (5, 1832–1951)**: Hudson Taylor, Jonathan Goforth, Amy Carmichael, Evan Roberts, Jessie Penn-Lewis
- **Tier 6 — Modern & Chinese Church (7, 1885–1991)**: T. Austin-Sparks, A. W. Tozer, Martyn Lloyd-Jones, Dietrich Bonhoeffer, Watchman Nee, Wang Mingdao, Song Shangjie

All source texts are in the public domain, sourced primarily from Project Gutenberg, Internet Archive, CCEL, and austin-sparks.net (the latter explicitly waived to public domain by the author). The largest single corpora are Madame Guyon (5.9 MB EN + 3.3 MB ZH, 31 works), John Calvin (7.7 MB EN), Augustine (3.3 MB EN + 0.7 MB ZH), and T. Austin-Sparks (2.0 MB EN + 1.2 MB ZH, scraped from 12 books / 106 chapters).

These texts use a distinctive vocabulary — “dark night,” “interior castle,” “practicing the presence,” “abiding in Christ,” “得胜者”, “破碎”, “宇宙性的十字架” — that is specific enough for keyword search to work reliably. When a user asks about “the dark night of the soul,” `grep` finds exactly the right passages. When a Chinese-speaking user asks 灵魂的暗夜, the Layer 2 concept files (compiled in Chinese from English originals) bridge the gap.

4.3 USCIS Civics (128 questions)

The civics corpus consists of the official 128 USCIS naturalization test questions with their approved answers. This is a closed, well-defined knowledge set that changes infrequently. Each question-answer pair is stored as a discrete entry, and any keyword from the question retrieves the corresponding answer with 100% reliability.

---

5. Comparative Analysis

We compare Knowledge Search against two established paradigms: Vector RAG (the standard embedding + vector database approach) and GraphRAG (Microsoft's graph-based retrieval system).

Dimension	Knowledge Search	Vector RAG	GraphRAG
Retrieval Accuracy	100%	~85-95%	~90-95%
Query Latency	<10ms	50-200ms	100-500ms
Preprocessing Time	0	Hours	Hours
Additional Memory	0	500MB+	1GB+
Infrastructure Dependencies	None	Vector DB + Embedding API	Graph DB + Embedding API + LLM for extraction
Maintenance on Corpus Update	Drop file in directory	Re-embed and re-index	Re-extract entities, re-build graph
Failure Modes	Query contains no domain keywords	Embedding drift, chunk boundary splits, ANN approximation errors	Entity extraction errors, incomplete graph, relationship hallucination
Explainability	Trivial (exact match + line number)	Low (embedding similarity score)	Medium (graph traversal path)
Cost	$0	Embedding API calls + DB hosting	Embedding + LLM extraction + DB hosting
Lines of Code	~30	~300-500	~1000+

5.1 On Accuracy

The 100% accuracy claim for Knowledge Search requires qualification. We define accuracy as: *given a query that contains at least one domain-relevant keyword, does the system retrieve all and only the relevant passages?* Under this definition, `grep` achieves perfect recall and high precision — it finds every occurrence of the search term and returns only passages containing it.

Vector RAG's accuracy gap stems from multiple sources: embedding model limitations for specialized vocabulary, chunk boundary artifacts, and ANN approximation errors. In our testing with the TCM corpus, vector RAG consistently struggled with Classical Chinese terms that have no close equivalent in the embedding model's training data. The query "麻黄汤" (Mahuang Decoction) would sometimes retrieve passages about "桂枝汤" (Guizhi Decoction) because the two formulas share overlapping ingredient discussions and thus have similar embeddings.

GraphRAG's accuracy is higher than vector RAG for relationship-heavy queries but suffers from entity extraction errors — particularly for Classical Chinese texts where named entity recognition models perform poorly.

5.2 On Latency

The latency difference is not marginal. Knowledge Search completes in 2-8ms on the small civics corpus and 8-25ms on the full 180 MB / 500-file corpus — the time for a filesystem `grep` across all source and `_compiled/` files. Vector RAG requires an embedding API call (20-100ms for remote, 10-50ms for local), followed by an ANN search (5-20ms), followed by optional re-ranking (20-100ms). GraphRAG adds graph traversal on top of these costs.

For interactive agents that make multiple knowledge retrievals per conversation turn, the cumulative latency difference is significant. A TCM diagnostic agent that cross-references herbs, formulas, and symptoms might make 3-5 retrieval calls per turn. At 5ms each, Knowledge Search adds 25ms. At 100ms each, vector RAG adds 500ms — a delay the user can perceive.

5.3 On Operational Simplicity

This is where the difference is starkest. Adding a new text to Knowledge Search requires one operation: copy the file into the knowledge directory. There is no re-indexing, no re-embedding, no schema migration. The file is immediately available for search on the next query.

Adding a new text to a vector RAG system requires: reading the file, chunking it according to the configured strategy, embedding each chunk via the embedding model, inserting the vectors into the database, and verifying the index. If the embedding model has been updated since the last indexing run, the entire corpus should be re-embedded for consistency.

---

6. Why It Works

Knowledge Search works because of a property shared by most domain-specific knowledge bases: **predictable vocabulary**.

6.1 Vocabulary Predictability in Specialized Domains

Medical texts do not use creative synonyms. When a TCM text discusses Astragalus root, it says 黄芪. It does not say “that yellowish root that boosts energy” or “the immune-enhancing legume.” The vocabulary is standardized by millennia of scholarly convention.

The same property holds for spiritual texts (writers consistently use “contemplation,” “union with God,” “dark night”), legal texts (specific statute numbers, legal terms of art), and technical documentation (API names, error codes, configuration parameters).

This vocabulary predictability means that keyword search is not a crude approximation — it is the optimal retrieval strategy. The user’s query and the relevant passage share literal string overlap. No semantic interpretation is needed because the domain vocabulary is already precise.

6.2 Bounded Corpus Size

Knowledge Search is designed for corpora that are large enough to exceed LLM context windows but small enough for filesystem grep to be fast. Our production corpus totals approximately **180 MB** across **~500 source files plus ~2.7 MB of compiled Layer-2 entries**. `grep` searches the combined index in 8-25ms on commodity hardware (Apple Silicon Mac mini).

This is not a limitation — it is a realistic description of most domain-specific knowledge bases. A medical practice's clinical guidelines, a law firm's case files, a company's internal documentation: these are typically measured in tens to hundreds of megabytes, not terabytes. The scaling properties of vector databases are irrelevant at these sizes. The 3.6× growth in our own corpus over 17 days (from ~50 MB / 21 agents at the original draft of this paper to 180 MB / 76 agents at v1.1) required no architectural change.

6.3 The LLM as Semantic Layer

The critical insight is that the LLM itself provides the semantic understanding that vector RAG attempts to encode in the retrieval layer. When `grep` returns eight lines of context around a match for 黄芪, the LLM reads those lines and understands the relationships, implications, and nuances that an embedding model would only approximate.

By keeping the retrieval layer dumb and exact, we avoid the failure mode where the retrieval system's "intelligence" disagrees with the LLM's understanding. There is no semantic gap between what was retrieved and what the LLM interprets, because the LLM is doing all the interpretation on raw text.

6.4 Layer 2 as Concept Bridge, Not Fallback

A subtle but crucial property emerges from compiling per-source `_concepts.md` and `_faq.md` files in the user's likely query language: **the compiled Layer 2 acts as a multilingual semantic router back into the monolingual Layer 1 corpus**. When a Chinese-speaking user asks about a concept whose original text exists only in English, the query matches the Chinese phrasing in the Layer 2 file, the LLM reads the cited source attribution, then re-issues a follow-up grep against the original English chapter. The LLM completes the bridge that the embedding model would have attempted to short-cut.

Empirically this is what makes the system feel like it understands cross-lingual queries while every actual retrieval operation remains a literal `grep`.

6.5 Reproducibility Addendum: Failure and Recovery (2026-04-25)

Position papers are improved by including failure data. We document one such cycle.

The failure. On 2026-04-25 we issued the following query to one of our spiritual agents (slug `austin_sparks`, persona of T. Austin-Sparks, 1885–1971): *"对'破碎'您怎么看？请用中文回答，引用您原书的话。"* ("What is your view of 'brokenness'? Please answer in Chinese, quoting from your books.")

The agent returned a fluent, well-attributed answer containing **five direct quotes**, each with book name and chapter number:

Quote	Claimed source
"未经破碎的器皿无论多大，都把基督装在自己的形状里"	*The School of Christ*, Ch. 1
"破碎了，主自己的水才能流过来"	(same)
"十字架不只是赎罪的祭坛，是宇宙性的属灵原则"	*Centrality and Universality of the Cross*, Ch. 2
"我们被带进一种情形，在那里我们不能再凭自己作什么"	*We Beheld His Glory*
"主的手是慈爱的手，但祂的手也是破碎的手"	*The Arm of the Lord*

We then ran a literal `grep -rin` against the corresponding source files in `input/spiritual_en/austin_sparks/` and `input/spiritual_zh/austin_sparks/`. **Zero of the five quotes appeared in the corpus.** The fluent attributions were fabrications.

Root cause analysis. The retrieval system was not at fault — `grep` and `cat` returned exactly what they were asked to. The fault was upstream, in the soul prompt itself. The persona file `austin_sparks.soul.md` contained authorial signature phrases written by the prompt author as voice-matching guidance, formatted as `**"..."**` (bold + quotation marks). Examples:

```
**管道与器皿**：神的仆人不是表演者，是透明的管道。
"破碎了，主的水才能流过来"

一个未经破碎的器皿，无论恩赐多大，**都把基督装在自己的形状里**
```

The LLM, faced with a request to "quote from your books," parsed these as canonical text written by Austin-Sparks himself, then attached plausible chapter numbers to lend the fabrications structural authority. This is a known failure mode in instruction-tuned models when the prompt itself contains text formatted as quotation.

The fix. We applied two changes (commit `3e365a9`):

1. **Auto-strip soul-prompt fake quote markers across all 79 souls.** A 60-line Python script regex-matched `\*\*"([^"]+)"\*\*` and replaced with `*\1*` (italic only, no quotes), removing 41 instances across 13 soul files. The signature phrases survive as voice guidance but no longer wear the visual costume of canonical text.
2. **Append a "Citation Hard Rules" block to all 79 souls.** The block (under 200 words, marked `<!--citation-rule-v1 -->` for idempotent future regeneration) instructs the agent that any quoted text must be a literal substring returned by `knowledge_search` in the current turn, and that signature phrases from the soul prompt are voice guidance, not canonical text to be quoted.

Total time from diagnosis to deploy: ~**25 minutes**, including a fleet restart for soul reload.

The recovery. We reissued the identical query. The agent returned a new answer with **four quotes**:

Quote	Source claim	grep verification
"the vessel, thus wrought upon, is the message. People do not come to hear what you have to teach. They have come to see what you are"	*Prophetic Ministry*, Ch. 2	✅ `06_prophetic_ministry.txt:141`
"It is not that you have achieved something, but rather that you have been broken in the process"	*Prophetic Ministry*	✅ `06_prophetic_ministry.txt:167`
"有受破碎、受拆毁之心的人…才能真实得着释放"	*主的膀臂* (Chinese)	✅ `04_主的膀臂.txt:317`
"This brokenness, helplessness, hopelessness, and yet faith"	*Centrality of the Cross*, Ch. 2	✅ `04_centrality_universality_cross.txt:421`

4/4 grep-verified. Citation accuracy moved from 0/5 to 4/4 in one reload cycle.

Architectural implications. Three observations follow.

First, the zero-hallucination contract of Knowledge Search is **architecturally guaranteed** in the sense that any text wearing quote-marks must, after the citation hard-rule, correspond to a `grep`-locatable substring of an actual file. Drift is detectable by re-running `grep` against the agent's output. There is no equivalent test in vector RAG: a retrieved chunk that has been paraphrased by the LLM cannot be unambiguously traced to source.

Second, **the failure was upstream of retrieval and the recovery was upstream of retrieval**. No model was retrained; no embedding was recomputed; no infrastructure was changed. A 60-line Python script and a 200-word prompt addendum, applied to text files on disk, restored the safety property. This is the cost-of-correction profile of a system whose intelligence lives in plain text.

Third, the failure is **reproducible and auditable**: the bad commit, the diagnostic grep, the fix commit, and the post-fix grep are all recorded as git history in the public LocalKin repositories. Researchers wishing to replay the cycle can `git checkout` either side of commit `3e365a9` and reproduce both halves of the experiment with the same query.

We include this section because we believe the most useful systems papers honestly report not just the working mode but the failure-and-recovery mode, with timestamps. Architectures that cannot be diagnosed and patched on this timescale are architectures that cannot be trusted.

---

7. Limitations and Honest Boundaries

We do not claim Knowledge Search is a universal replacement for vector RAG. It has clear limitations that practitioners should understand.

7.1 Open-Domain General Knowledge

Knowledge Search requires a bounded corpus with predictable vocabulary. It is not suitable for open-domain question answering where the relevant information could appear in any text using any vocabulary. A general-purpose chatbot that needs to answer questions about arbitrary topics should use vector RAG or web search.

7.2 Semantic Similarity Search

When the user's intent cannot be expressed as any keyword in any language — "texts that discuss a vague longing for transcendence" — `grep` will not help. Vector RAG's ability to match semantic similarity, despite its imprecisions, is genuinely valuable for this class of open-ended conceptual queries.

Our mitigation (Layer 2 per-source LLM-compiled concept and FAQ files) substantially closes this gap by **pre-translating concepts into the user's likely query language during nightly compilation**. As of 2026-04-25, 350/510 sources have such bilingual companion files. The remaining gap — truly open-ended semantic queries that match no concept name in any author's vocabulary — is real but smaller than it appears in casual analysis. We have not yet found a production query in the spirituality or TCM domains that grep + Layer 2 fails to handle.

7.3 Cross-Lingual Retrieval Without Shared Vocabulary

Our TCM corpus is primarily Chinese and our spiritual corpus is bilingual (Chinese where translations exist, otherwise English). A query in one language about a concept whose source text exists only in the other language is bridged by Layer 2: the LLM-compiled `_concepts.md` and `_faq.md` files for each source are generated in the language detected from the source filename and content, but cross-language Q&A is increasingly the norm. The 0/5 → 4/4 case in §6.5 is itself a cross-lingual experiment: a Chinese query against a primarily English corpus, recovering correct citations from English originals.

That said, Layer 2 cross-lingual coverage at first compilation depends on the LLM's bilingual training. We chose Kimi 2.6 (`kimi-k2.6:cloud`, served via local Ollama) over Anthropic Haiku 4.5 partly because Kimi's training corpus appears richer in Chinese theological and TCM vocabulary, producing higher-fidelity concept extracts on these domains.

7.4 Very Large Corpora

At corpus sizes beyond ~1GB, filesystem `grep` latency becomes noticeable. At 10GB+, it becomes impractical for interactive use. Vector databases with pre-built indexes maintain sub-100ms query times regardless of corpus size. For truly large-scale knowledge bases, the infrastructure overhead of vector RAG is justified by the scaling requirements.

---

8. Production Integration

Knowledge Search is not a standalone system — it is a skill within the LocalKin multi-agent platform, invoked by agents as needed during conversation.

8.1 Agent Integration

The `knowledge_search` skill exposes a simple interface: given a query string and a knowledge domain, return matching passages from both Layer 1 raw sources and Layer 2 compiled concepts/FAQ. As of 2026-04-25 it is used by:

- **39 TCM master agents** (deployed at `heal.localkin.ai`): Yellow Emperor, Zhang Zhongjing, Hua Tuo, Sun Simiao, Li Shizhen, Huang Yuanyu, Ye Tianshi, Liu Lihong, Ni Haixia, and 30 others spanning 4,500 years.
- **37 spiritual master agents** (deployed at `faith.localkin.ai`): Irenaeus, Augustine, Thomas à Kempis, Madame Guyon, Martin Luther, John Calvin, John Bunyan, George Müller, Hudson Taylor, T. Austin-Sparks, A. W. Tozer, Watchman Nee, Wang Mingdao, and 24 others spanning 1,900 years (130 AD – 1991 AD).
- **1 citizenship coach**: queries the USCIS civics corpus for naturalization test preparation.

A single diagnostic turn for Zhang Zhongjing's agent might involve three sequential knowledge searches: one for the presenting symptom pattern, one for the relevant herbal formula, one for contraindications. Total retrieval time: ~15-30ms across the now-larger corpus. The agent's response generation (LLM inference, served by `kimi-k2.5:cloud` Ollama for the master persona, with Claude Haiku 4.5 as fallback) takes 2-5 seconds. Retrieval is never the bottleneck.

Multi-master debate (cross-fleet) is implemented as parallel `streamChat` calls against multiple agents, each independently invoking `knowledge_search` against its own corpus directory. Two agents in a debate never share retrieval state; each grounds in its own master's writings. Architecturally this is enabled by the per-author corpus directory layout — there is no shared index that needs to be partitioned by author.

8.2 Skill Implementation

The retrieval-side skill remains approximately 30 lines of shell. The core logic:

```
# Single grep walks both raw sources and _compiled/ concept+FAQ files
# The agent never knows which layer matched — it just sees passages.
results=$(grep -r -i -n -C 8 "$query" "$KNOWLEDGE_DIR" 2>/dev/null)
echo "$results"
```

The compilation-side skill (`knowledge_compile.py`, ~400 lines of Python) is more substantial but runs only at scheduled times, not at query time. It exposes four actions:

Action	Purpose
`status`	Report compilation coverage per author + per domain
`list_needed`	List source files awaiting compilation
`compile`	Compile one specified source file
`compile_author`	Compile all source files for one author
`compile_batch`	Compile up to `--limit N` uncompiled files across a domain

There is no configuration file at query time. There is no service to start. There is no embedding model to load. The skill works on any Unix-like system with a filesystem and Python 3.10+; `curl` is the only network call (to localhost Ollama).

---

9. Autonomous Corpus Growth

A common objection to our approach is that it requires manual corpus curation, and that the Layer 2 concept/FAQ files require manual or paid LLM compilation. We address both objections with a fully autonomous, free, daily compilation pipeline.

9.1 Source-File Acquisition

New raw texts are added by copying them into the appropriate knowledge directory:

```
input/spiritual_en/<slug>/<filename>.txt
input/spiritual_zh/<slug>/<filename>.md
input/tcm_zh/<slug>/<filename>.md
```

There is no re-indexing step, no re-embedding step, no pipeline to trigger. The next `grep` query immediately finds the new file. In the past 30 days the system has absorbed 14 new master personas and ~50 source files this way, including a one-day acquisition of 12 books / 106 chapters of T. Austin-Sparks corpus from `austin-sparks.net` via standard `curl` + a 100-line Python scraper.

9.2 Layer 2 Cron-Based Compilation

The Layer 2 `_compiled/` directory grows automatically via three nightly cron entries (in `~/.localkin/cron.yaml`):

```yaml
- name: "knowledge-growth-spiritual-en"
  cron: "0 5 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain spiritual_en --limit 30"

- name: "knowledge-growth-spiritual-zh"
  cron: "30 5 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain spiritual_zh --limit 30"

- name: "knowledge-growth-tcm-zh"
  cron: "0 6 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain tcm_zh --limit 30"
```

The 30-minute stagger between the three jobs avoids concurrent load on the local Ollama instance serving Kimi 2.6. Each job processes up to 30 uncompiled source files per night, generating both `<source>_concepts.md` and `<source>_faq.md` per file. At ~24 seconds per file (two LLM calls), each job completes in ~12 minutes, totalling ~36 minutes of Ollama work per night.

Cost analysis. Migration from Anthropic Haiku 4.5 (paid API) to Kimi 2.6 (local Ollama) reduced per-file cost from ~$0.01 to **$0.00**. At 90 files/night sustained, the previous regime would have cost ~$0.90/day or ~$330/year. The current regime costs **$0/year** in API fees; electricity for the always-on Mac mini is unmetered.

9.3 Empirical Growth Curve

The architecture's "scales without re-architecting" claim is empirically supported by the system's own growth in 17 days:

Date	Agents	Source files	Layer 2 compiled	Notes
2026-04-08 (paper v1.0)	21	~162	~30 (NotebookLM, manual)	One-shot manual compilation
2026-04-21	64	~250	~47	After Wave 1 spiritual expansion
2026-04-24	73	~480	~135	Mid catch-up sweep
2026-04-25 (paper v1.1)	**76**	**~510**	**~350 (68%)**	After cron migration to Kimi 2.6

The 3.6× growth in agent count and 3.2× growth in corpus size required no architectural changes: no schema migration, no embedding model retraining, no infrastructure provisioning. The same `grep` invocation works against the larger corpus with the same code path.

9.4 Failure Modes Observed in Production

We document the failure modes encountered during this 17-day growth, which a vector RAG system would have manifested differently:

1. **Alphabetical port-shift breakage (2026-04-24).** Adding three new spiritual masters (Kempis, de Sales, Austin-Sparks) shifted alphabetical port assignments across the fleet, pushing four TCM agents above the gateway's discovery range ceiling (port 9350). Symptom: `unknown agent` errors. Fix: bump ceiling to 9450 in one line of Go (`fleetPorts = [][2]int{{9100, 9450}}`) and rebuild gateway. Time-to-fix: 3 minutes including restart. There is no equivalent failure mode in vector RAG because there is no per-agent corpus partition.
2. **Soul-prompt fake-quote regression (2026-04-25).** Documented in detail in §6.5 above. Time-to-fix: 25 minutes.
3. **Mislabeled author corpus (2026-04-24).** The `ni_haixia/` directory had been seeded with reading-notes about other authors' books, not Ni Haixia's own teaching. Symptom: agent answers using third-party content but attributed to Ni Haixia. Fix: replace with 6 actual Renji series transcripts (5.8 MB) from a public-domain GitHub repository; rerun `compile.py --action compile_author --author ni_haixia`. Time-to-fix: 1 hour, mostly download time. The retrieval architecture made the diagnosis obvious — `grep -l` immediately revealed the false attributions.
4. **Middleware slug drift (2026-04-24).** The web frontend's middleware was hard-coding the master slug list, which drifted behind the live `masters.ts` source after Wave 1+2 expansion. Symptom: 10 master URLs returned 404. Fix: change middleware to import the slug list from `masters.ts` directly (`-75 lines of duplication`). This is not a Knowledge Search failure, but it illustrates the broader pattern of "drift between source-of-truth and shadow copies" that this paper's architecture is designed to avoid.

In each case the root cause was diagnosable by reading source files and grepping. Recovery did not require model retraining, embedding recomputation, or vector store reindexing. We submit this as quiet evidence for the paper's central claim: when the system's intelligence lives in plain text, the system's repair lives in plain text.

---

10. Discussion: Retrieval Doesn't Need Intelligence

The machine learning community has a tendency to solve every problem with more machine learning. Retrieval is a case study in this tendency. The progression from BM25 to dense retrieval to learned sparse retrieval to multi-vector retrieval to GraphRAG represents increasing model complexity applied to the retrieval layer — each step adding parameters, training data requirements, and infrastructure dependencies.

We suggest this progression has overshot for a large class of practical applications. When the knowledge base is domain-specific, the vocabulary is predictable, and the corpus is bounded, the optimal retrieval system is the one that has been available since 1973: `grep` (Thompson, 1973).

This is not an argument against embeddings or vector databases in general. It is an argument against the reflexive application of complex systems to problems that do not require them. The engineering decision should be: *does my retrieval problem require semantic understanding at the retrieval layer, or can I defer that understanding to the LLM?*

For the majority of domain-specific grounding tasks we have encountered — and we have deployed agents across medicine, spirituality, and civics — the answer is: defer it to the LLM. Let retrieval be fast, exact, and dumb. Let the LLM be the intelligence.

10.1 Implications for the Field

If our findings generalize — and we believe they do, for the class of problems described — then the standard advice to "just use RAG" deserves significant qualification. Practitioners building domain-specific LLM agents should consider keyword search as a first approach, not a last resort. The burden of proof should be on the complex system to justify its complexity, not on the simple system to justify its simplicity.

10.2 The Broader Pattern

Knowledge Search is an instance of a broader pattern we observe in LLM application development: **the best systems are often the ones that let the LLM do more and the infrastructure do less.** Sophisticated retrieval, complex orchestration, elaborate prompt chains — these are often symptoms of underestimating the LLM's ability to handle messy, unstructured input.

Give the model the raw text. Give it enough context. Get out of the way.

---

11. Conclusion

We have presented Knowledge Search, a two-layer retrieval system that replaces the standard vector RAG pipeline with `grep` over raw text plus `grep` over LLM-compiled per-source concept and FAQ files. Deployed across **76 specialized LLM agents** serving three knowledge domains with **~500 primary source texts (~180 MB)**, it achieves 100% retrieval accuracy at 8-25ms latency with zero per-query preprocessing, zero infrastructure dependencies at query time, and approximately 30 lines of retrieval-side implementation code (the Layer 2 compilation skill is ~400 lines and runs only on a nightly cron at $0/night).

The system works because domain-specific knowledge bases have predictable vocabulary, bounded size, and deterministic search requirements — properties that make keyword search not merely adequate but optimal. The semantic understanding needed to synthesize retrieved passages into useful answers is provided by the LLM itself, making intelligent retrieval redundant. The Layer 2 concept-and-FAQ files, automatically generated by a free local LLM, provide a multilingual semantic bridge into the literal Layer 1 corpus without any embedding store.

We have additionally documented one full failure-and-recovery cycle (§6.5): a deliberate prompt-engineering regression collapsed citation accuracy to 0/5 grep-verified quotes; a 25-minute fix (a 60-line Python script and a 200-word soul-prompt addendum) restored it to 4/4. The architecture's safety properties recover through prompt hygiene alone — no retraining, no infrastructure change. We submit this as a stronger form of "100% retrieval accuracy" than is typical in retrieval papers: not just a benchmark number, but a reproducible recovery path.

We do not claim this approach replaces vector RAG for all applications. We claim it replaces vector RAG for more applications than the current consensus assumes — and that the autonomous nightly compilation pipeline closes most of the gap on the cross-lingual and conceptual queries where pure keyword search was previously inadequate. Before reaching for embeddings, vector databases, and approximate nearest neighbor search, ask:

would `grep` work — and if not, would `grep` plus a nightly cron of LLM-compiled concept files work? You might be surprised how often the answer to one of these is yes.

References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpathy, A., Goyal, N., … & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

Thompson, K. (1973). The UNIX command language. *Structured Programming*, Infotech State of the Art Report, 375-384.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Robertson, S. E., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., … & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., … & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769-6781.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. (Cited as a related work on self-correcting LLM agents — the prompt-hygiene fix in §6.5 is a non-RL instance of the same recovery pattern.)

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … & Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

Correspondence: The LocalKin Team — `contact@localkin.ai`. This paper describes the knowledge retrieval system deployed in LocalKin (https://localkin.dev) as of v1.1.0 / 2026-04-25. Reproduction artifacts (souls, scripts, and the failure-and-recovery git history of §6.5) are public at https://github.com/LocalKinAI.

"Grep is All You Need" is a deliberate homage to Vaswani et al. (2017). We trust the irony is not lost.

Cite as:

```bibtex
@misc{localkin2026grep,
  author    = {{The LocalKin Team}},
  title     = {Grep is All You Need: Zero-Preprocessing Knowledge
               Retrieval for LLM Agents},
  year      = {2026},
  month     = apr,
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.19777260},
  url       = {https://doi.org/10.5281/zenodo.19777260},
  note      = {Correspondence: contact@localkin.ai;
               code at https://github.com/LocalKinAI/grep-is-all-you-need}
}
```

Grep 即是你所需要的一切：面向 LLM 智能体的零预处理知识检索

The LocalKin Team *通讯：* `contact@localkin.ai` *项目：* https://localkin.dev | https://github.com/LocalKinAI

立场论文 — 2026 年 4 月（v1.1，更新于 2026-04-25） DOI: 10.5281/zenodo.19777260

摘要

检索增强生成（RAG）已成为将大型语言模型（LLM）智能体基于领域特定知识的主流范式。标准方法需要选择嵌入模型、设计分块策略、部署向量数据库、维护索引，以及在查询时执行近似最近邻（ANN）搜索。我们认为，对于领域特定知识基础化——词汇可预测且语料库有界的场景——整个技术栈是不必要的。我们提出*知识搜索*（Knowledge Search），一个由（1）对原始源文本带上下文行窗口的 `grep`，和（2）对**每源文件 LLM 编译生成的概念与 FAQ 文件**（由免费的本地自主每夜编译流水线产出）的 `grep` 组成的双层检索系统。该系统在生产环境中部署于服务三个知识领域（传统中医、基督教灵修经典和美国公民知识）的 **76 个专业 LLM 智能体**中，基于约 **500 份原始文献和约 180 MB 语料库**，实现了 100% 检索准确率、不到 10ms 的延迟、查询时零预处理、零额外内存占用和零基础设施依赖。我们另记录了一次可复现的失败-恢复周期（0/5 杜撰引文 → 一次性提交修复后 4/4 grep 验证），证明该架构的安全属性可以仅通过提示卫生即可恢复——无需重训，无需基础设施变更。关键洞见很简单：检索不需要智能。LLM 才是智能。

关键词： 检索增强生成、知识基础化、LLM 智能体、信息检索、领域特定 AI、零幻觉检索、自主语料库增长

---

1. 引言

2026 年，每个 LLM 应用教程都以同样的方式开始：选择嵌入模型、分块文档、启动向量数据库、构建索引，然后祈祷近似最近邻搜索能返回正确的段落。这个流水线——统称为检索增强生成（Lewis et al., 2020）——已变得如此普遍，以至于被视为自然法则，而非它实际所是的：一个具有重大权衡的工程选择。

我们提出一种替代方案。对于领域特定知识基础化，在源文本已知、词汇可预测且语料库在合理范围内的情况下，整个 RAG 技术栈可以被两个早于万维网的 Unix 工具替代：`grep` 和 `cat`。

这不是玩具实验。我们的系统*知识搜索*作为 LocalKin（一个多智能体 AI 平台）的一部分部署在生产中。截至 2026 年 4 月，它作为 **39 个传统中医（TCM）智能体、37 个基督教灵修方向智能体**和一个美国公民辅导智能体的知识骨干——共 **76 个专业智能体**（免费层白名单），基于跨越两种语言、四千五百年人类思想的约 **500 份原始文献（约 180 MB）**（从黄帝内经到健在的国医大师；从爱任纽 130 AD 到史百克 1971）。整个系统由一台 Mac mini 服务全部 76 个智能体。

结果毫不接近。知识搜索在不到 10ms 的查询时延下实现 100% 检索准确率且零查询时预处理，而向量 RAG 系统在数小时前置预处理后通常提供 85-95% 的准确率和 50-200ms 的延迟。我们不主张这种方法适用于一切。我们主张，对于大多数实践者反射性地伸手抓向量数据库的那类问题，它的效果出奇地好。

而且，该架构的"零幻觉"属性不仅是愿景——它是可复现的。我们记录了一次单日周期（第 6.5 节）：一次刻意的提示工程倒退使引文准确率崩溃为 0/5 grep 验证；通过批量剥离 79 个 soul 提示中 41 处假引文标记 + 加入引用硬约束，在 25 分钟内恢复至 4/4——仅靠提示卫生，无需重训，无需基础设施变更。

本文结构如下。第 2 节检视标准 RAG 流水线的隐性成本。第 3 节呈现我们的双层检索架构。第 4 节描述知识语料库。第 5 节提供比较分析。第 6 节解释这种方法为什么有效，并包含可复现性附录（§6.5）。第 7 节诚实地处理其局限性。第 8 节讨论生产集成。第 9 节涵盖自主语料库增长——一条由每日 cron 驱动的流水线，已在 17 天内零成本编译了 47→345 个概念/FAQ 条目。第 10 节反思这对该领域意味着什么。

---

2. 向量 RAG 的隐性成本

标准 RAG 流水线被呈现为已解决的问题，但每个阶段都引入了复合的复杂性、延迟，以及——最关键的——信息损失。

2.1 嵌入模型选择

第一个决策是使用哪个嵌入模型。OpenAI 的 `text-embedding-3-large`？Cohere 的 `embed-v3`？微调的 Sentence-BERT 变体？每个模型编码不同的语义假设。主要在英文网络文本上训练的模型将为古典汉语医学术语产生糟糕的嵌入。这个选择是有实质影响的，然而没有原则性的方法来做出它，除了进行大量评估——而评估需要你尚未构建的检索系统本身。

2.2 分块策略

文档必须在嵌入前分割成块。但怎么分？512 个 Token 的固定大小窗口？按标题递归分割？基于主题边界的语义分块？每种策略都是原始文本的有损压缩。关于黄芪的段落如果跨越块边界，将被分成两个片段，两者都不能完整捕获原始含义。块大小直接决定检索质量的上限，但必须在任何检索发生之前选择。

2.3 向量数据库操作

嵌入的块必须存储在向量数据库中——Pinecone、Weaviate、Chroma、Qdrant、pgvector，或 2023 年以来涌现的数十种替代品之一。每种都需要自己的部署、配置和操作专业知识。每种都有不同的一致性保证、扩展特性和故障模式。对于独立开发者或小团队来说，这不是微不足道的依赖——它是一个必须监控、备份和维护的完整子系统。

2.4 近似最近邻搜索

在查询时，用户的问题被嵌入，并使用近似最近邻算法（HNSW、IVF 或类似算法）与存储的向量进行比较。"近似"这个词在这里承担了很多工作。ANN 搜索以准确性换速度，这个权衡并不总是有利的。关于伤寒论的查询可能会检索到关于温病的段落，因为嵌入在几何上很接近——两个文本讨论重叠的症状。检索到的段落看似合理但是错误的，LLM 无法知道这一点。

2.5 维护税

添加新文档时，索引必须重建。更新嵌入模型时，整个语料库必须重新嵌入。调整块大小时，一切重新开始。这种维护税在演示中是不可见的，但在生产系统的运营成本中占主导地位。

---

3. 我们的方法：双层知识检索

知识搜索用两层替换整个 RAG 流水线，每层作为单个系统调用实现。

3.1 第一层：grep——精确上下文搜索

第一层使用带 8 行上下文窗口的 `grep` 对原始源文本执行关键词搜索：

```
grep -r -i -n -C 8 "$query" "$knowledge_dir"
```

标志很直接： - `-r`：在知识目录中所有文件的递归搜索 - `-i`：不区分大小写匹配 - `-n`：包含行号以供来源归属 - `-C 8`：在每个匹配前后返回 8 行上下文

这并不复杂。这正是重点所在。当用户询问黄芪的功效时，搜索词黄芪会在 TCM 语料库的每个相关段落中逐字出现。没有嵌入可以误解，没有块边界可以分割答案，没有近似搜索可以返回接近命中但不准确的结果。匹配是精确的，上下文是完整的，检索是确定性的。

来自不同源文本的多个匹配被连接并传递给 LLM，LLM 综合出答案。LLM 看到的原始文本与书写时完全相同，周围上下文完整。没有信息丢失。

性能特征： - 延迟：在商品硬件上对 162 个文件的语料库为 2-8ms - 准确率：包含领域词汇的查询 100% 召回率 - 预处理：无 - 内存开销：无（文件保留在磁盘上，按需读取）

3.2 第二层：grep——每源文件 LLM 编译的概念与 FAQ 桥梁

并非每个查询都包含原始语料库语言中可 grep 的关键词。一个中文用户可能会问”史百克对’破碎’的看法”，而源语料库全是英文（`brokenness`、`broken vessel`、`the cross deals with the natural man`）。纯第一层 grep 在英文文本上是无法匹配中文查询词的。

第二层用一个**每源文件的概念-FAQ 编译层**解决这个问题，由一个每日 cron 任务自动生成。语料库中的每个源文件都维护两个小型伴随文件：

```
input/<domain>/<author>/                          # 第一层：原始原文
├── 01_school_of_christ.txt        (176 KB)
└── _compiled/                                     # 第二层：自动生成
    ├── 01_school_of_christ_concepts.md    (~3 KB, 核心概念 + 原文引用)
    └── 01_school_of_christ_faq.md         (~2.5 KB, 5–8 个 Q&A 对)
```

编译过程由 `knowledge_compile.py` 执行——一个 400 行的 Python skill，工作流程如下：

1. 读取一个源文件（截至 150 KB 以适配 LLM 上下文）
2. 向本地 LLM 发送两个 prompt：
   - **概念 prompt**：抽取 5–10 个核心概念，格式为 `<名称>: 定义 + 1 条关键原文引用`，加 3–5 条带章节归属的引用
   - **FAQ prompt**：以学生真实会问的方式生成 5–8 个 Q&A 对

3. 将输出写入 `_compiled/` 同级目录
4. 后续运行时若两个编译文件已存在则跳过（幂等）

关键实现选择（2026-04-24 修订）： 主要 LLM 从付费 API（Anthropic Haiku 4.5 via OAuth）迁移到 **本地 Ollama 部署的 Kimi 2.6**（`kimi-k2.6:cloud`）。迁移使每文件编译成本从约 $0.01 降至 **$0.00**，在中文灵修和中医内容上无质量退化。Haiku 保留作为 fallback，当本地 Kimi 实例返回空或过短输出时使用。完整调用点：

```
def call_kimi(prompt):
    payload = json.dumps({
        "model": "kimi-k2.6:cloud",
        "prompt": SYSTEM_PROMPT + "\n\n" + prompt,
        "stream": False,
        "options": {"temperature": 0.3, "num_predict": 4000},
    })
    return curl_post("http://localhost:11434/api/generate", payload)
```

调用 `knowledge_search` 时，**两层用同一次 `grep` 调用一起搜索**：

```
grep -r -i -n -C 8 "$query" "$knowledge_dir"   # 同时遍历原始文件与 _compiled/
```

这是相对常规"备用"框架的架构反转：第二层**不是降级路径**，而是**并行的概念桥梁**。当用户查询中包含一个仅出现在第二层的术语（例如某个中文概念名对应一个英文源），grep 命中 `_concepts.md` 或 `_faq.md` 文件，LLM 据此定位到原始源中的正确段落。智能体随后从第一层引用并标注章节。

规模估算： 500 个源文件下，每源约 5.5 KB × 500 ≈ 2.7 MB 的编译材料——相对 180 MB 原始语料库可忽略。`grep` 性能不变。

生成节奏： 每日 cron 任务在 5:00、5:30、6:00 AM 错开运行（避免本地 Ollama 并发负载），每个域每夜处理 30 个新源文件。截至 2026-04-25 系统已在 17 天内编译 345/505 条目（68%），节奏约 90 条目/夜，$0/夜。

3.3 设计原则

该架构体现了一个原则：**检索不需要智能；LLM 才是智能。**

向量 RAG 系统试图在检索层中构建智能——通过嵌入的语义理解、通过相似度分数的相关性排名、通过交叉编码器的重新排名。这是将工程努力应用于错误层。LLM 已经是流水线中最强大的语言理解系统。给它原始文本，让它做它最擅长的事情。

---

4. 知识语料库

知识搜索部署在三个不同的知识领域，每个领域具有不同的特征。截至 2026-04-25，生产语料库共约 **500 份原始文献，总计约 180 MB**：

领域	源文件数	第一层大小	第二层已编译（4-25）
中医（中文）	171	~93 MB	115 / 171 (67%)
基督教灵修（英文）	114	~56 MB	115 / 115 (100%)
基督教灵修（中文）	219	~25 MB	115 / 219 (52%)
中医（英文桩）	5	<1 MB	5/5
合计	**509**	**~180 MB**	**350 / 510 (69%)**

4.1 传统中医（171 份中文源文件，39 位大师智能体）

TCM 语料库包含从黄帝（约公元前 2500 年）到健在的国医大师（截至 2026 年仍在临床）的经典医学文本，组织为按历史层级分组的 39 位大师智能体名册：

- **第一层 古典圣手（16 位）**：黄帝、张仲景、华佗、皇甫谧、孙思邈、刘完素、张子和、李东垣、朱丹溪、李时珍、张景岳、吴鞠通、叶天士、王清任、黄元御、傅青主
- **第二层 民国大家（5 位）**：张锡纯、曹颖甫、陆渊雷、蒲辅周、丁甘仁
- **第三层 经方现代复兴（4 位）**：胡希恕、刘渡舟、黄煌、范中林
- **第四层 当代国医大师（10 位）**：邓铁涛、朱良春、焦树德、颜德馨、周仲瑛、王琦、路志正、任继学、干祖望、裘沛然
- **特别派 / 当代（4 位）**：郑钦安（火神派）、刘力红（《思考中医》）、倪海厦、郝万山

基础著作包括《黄帝内经》（约 9 个源文件）、《伤寒论》（18 个版本）、《本草纲目》（5.2 MB）、《千金方》/《千金翼方》（孙思邈，约 11 MB）、《四圣心源》（黄元御，10 卷）、当代《人纪》系列文字稿（倪海厦，6 个讲座文本/5.8 MB）。

这些文本以古典汉语书写，医学词汇高度标准化。“气虚”这个术语两千年来一直意味着同样的事情。它不需要语义解读——它需要精确检索。

4.2 基督教灵修经典（333 份中英源文件，37 位大师智能体）

灵修语料库涵盖 1,900 年的默观性和神秘性基督教文学，组织为按六个层级分组的 37 位大师智能体：

- **第一层 教父时代（4 位，130–430 AD）**：爱任纽、亚他那修、金口约翰、奥古斯丁
- **第二层 默观神秘主义（9 位，14c–1897）**：《不知之云》作者、肯培多默、大德兰、十字若望、劳伦斯弟兄、莫利诺斯、方济各·沙雷氏、盖恩夫人、小德兰
- **第三层 改革与清教（3 位，1483–1688）**：马丁路德、约翰·加尔文、本仁约翰
- **第四层 大觉醒与复兴（9 位，1700–1898）**：辛生道夫、爱德华兹、卫斯理、怀特腓、芬尼、司布真、慕迪、慕安德烈、慕勒
- **第五层 宣教与世纪复兴（5 位，1832–1951）**：戴德生、古约翰、賈艾梅、罗伯斯、宾路易师母
- **第六层 现代与中国教会（7 位，1885–1991）**：史百克、陶恕、钟马田、潘霍华、倪柝声、王明道、宋尚节

所有原始文本均为公版，主要来源于 Project Gutenberg、Internet Archive、CCEL 与 austin-sparks.net（最后者由作者明确公开放弃版权）。最大单体语料是盖恩夫人（5.9 MB EN + 3.3 MB ZH，31 部作品）、约翰·加尔文（7.7 MB EN）、奥古斯丁（3.3 MB EN + 0.7 MB ZH）和史百克（2.0 MB EN + 1.2 MB ZH，从 12 部书 / 106 章爬取）。

这些文本使用独特的词汇——“dark night”（黑夜）、“interior castle”（内在城堡）、“practicing the presence”（练习临在）、“abiding in Christ”（住在基督里）、“得胜者”、“破碎”、“宇宙性的十字架”——足够具体使关键词搜索可靠工作。当英文用户询问 “the dark night of the soul”，`grep` 精确找到正确段落。当中文用户问”灵魂的暗夜”，第二层概念文件（由英文原文编译为中文）即承担起跨语言桥梁。

4.3 USCIS 公民知识（128 个问题）

公民知识语料库由官方 128 个 USCIS 入籍测试问题及其批准答案组成。这是一个封闭的、定义明确的知识集，不经常变化。每个问答对作为独立条目存储，问题中的任何关键词都能以 100% 可靠性检索到相应答案。

---

5. 比较分析

我们将知识搜索与两种已建立的范式进行比较：向量 RAG（标准嵌入 + 向量数据库方法）和 GraphRAG（微软的基于图的检索系统）。

维度	知识搜索	向量 RAG	GraphRAG
检索准确率	100%	~85-95%	~90-95%
查询延迟	<10ms	50-200ms	100-500ms
预处理时间	0	数小时	数小时
额外内存	0	500MB+	1GB+
基础设施依赖	无	向量 DB + 嵌入 API	图 DB + 嵌入 API + LLM 提取
语料库更新时的维护	将文件放入目录	重新嵌入和重新索引	重新提取实体，重建图
故障模式	查询不含领域关键词	嵌入漂移、块边界分割、ANN 近似误差	实体提取错误、图不完整、关系幻觉
可解释性	简单（精确匹配 + 行号）	低（嵌入相似度分数）	中（图遍历路径）
成本	$0	嵌入 API 调用 + DB 托管	嵌入 + LLM 提取 + DB 托管
代码行数	~30	~300-500	~1000+

5.1 关于准确率

知识搜索的 100% 准确率主张需要限定。我们将准确率定义为：*给定包含至少一个领域相关关键词的查询，系统是否检索所有且仅有相关段落？* 在此定义下，`grep` 实现完美召回率和高精确率——它找到搜索词的每个出现，并只返回包含它的段落。

向量 RAG 的准确率差距来自多个来源：专业词汇的嵌入模型限制、块边界问题和 ANN 近似误差。在我们用 TCM 语料库的测试中，向量 RAG 一致地在古典汉语术语上挣扎，这些术语在嵌入模型的训练数据中没有接近的等价物。查询”麻黄汤”有时会检索关于”桂枝汤”的段落，因为两个方剂共享重叠的配料讨论，因此具有相似的嵌入。

GraphRAG 的准确率对关系密集的查询高于向量 RAG，但受到实体提取错误的影响——特别是对于命名实体识别模型表现不佳的古典汉语文本。

5.2 关于延迟

延迟差异不是边缘性的。知识搜索在小型 civics 语料库上 2-8ms 内完成，在完整 180 MB / 500 文件语料库上 8-25ms 内完成——这是文件系统 `grep` 遍历所有源文件和 `_compiled/` 文件所需的时间。向量 RAG 需要嵌入 API 调用（远程 20-100ms，本地 10-50ms），随后是 ANN 搜索（5-20ms），随后是可选的重新排名（20-100ms）。GraphRAG 在这些成本之上增加了图遍历。

对于每次对话轮次进行多次知识检索的交互式智能体，累积延迟差异是显著的。每轮交叉参考草药、方剂和症状的 TCM 诊断智能体可能每轮进行 3-5 次检索调用。每次 5ms，知识搜索增加 25ms。每次 100ms，向量 RAG 增加 500ms——用户可以感知到的延迟。

5.3 关于操作简单性

这里的差异是最大的。向知识搜索添加新文本只需要一个操作：将文件复制到知识目录。没有重新索引、没有重新嵌入、没有 schema 迁移。文件在下一次查询时立即可供搜索。

向向量 RAG 系统添加新文本需要：读取文件、根据配置的策略分块、通过嵌入模型嵌入每个块、将向量插入数据库，以及验证索引。如果嵌入模型自上次索引运行以来已更新，整个语料库应重新嵌入以保持一致性。

---

6. 为什么有效

知识搜索有效，因为大多数领域特定知识库共享一个属性：**可预测的词汇**。

6.1 专业领域中的词汇可预测性

医学文本不使用有创意的同义词。当 TCM 文本讨论黄芪时，它说黄芪。它不说”那种能量提升的黄色根”或”免疫增强豆科植物”。词汇由数千年的学术惯例标准化。

同样的属性适用于灵修文本（作者一致使用”contemplation”默观、“union with God”与上帝合一、“dark night”黑夜）、法律文本（特定条款编号、法律术语）和技术文档（API 名称、错误代码、配置参数）。

这种词汇可预测性意味着关键词搜索不是粗糙的近似——它是最优的检索策略。用户的查询和相关段落共享文字串重叠。不需要语义解释，因为领域词汇本身已经精确。

6.2 有界语料库大小

知识搜索设计用于足够大以超过 LLM 上下文窗口但足够小以使文件系统 grep 快速的语料库。我们生产语料库总计约 **180 MB**，约 **500 份源文件加约 2.7 MB 编译后第二层条目**。`grep` 在 8-25ms 内搜索完合并索引（Apple Silicon Mac mini 商用硬件）。

这不是局限性——它是对大多数领域特定知识库的现实描述。医疗机构的临床指南、律师事务所的案件档案、公司的内部文档：这些通常以几十到几百兆字节计量，而非 TB。向量数据库的扩展属性在这些大小上是无关紧要的。我们自己的语料库 17 天内增长 3.6 倍（从论文初稿 v1.0 时约 50 MB / 21 智能体到 v1.1 的 180 MB / 76 智能体）未需任何架构变更。

6.3 LLM 作为语义层

关键洞见是 LLM 本身提供了向量 RAG 试图在检索层编码的语义理解。当 `grep` 为黄芪的匹配返回八行上下文时，LLM 阅读这些行并理解嵌入模型只能近似的关系、含义和细微差别。

通过保持检索层愚笨而精确，我们避免了检索系统的"智能"与 LLM 理解不一致的故障模式。检索到的内容和 LLM 解读之间没有语义差距，因为 LLM 正在对原始文本进行所有解读。

6.4 第二层作为概念桥梁，不是备用

一个微妙但关键的属性源于在用户可能的查询语言中编译每源文件 `_concepts.md` 与 `_faq.md`：**编译后的第二层充当一个多语言语义路由器，反向定位回单语言的第一层语料库**。当中文用户询问一个原文仅以英文存在的概念，查询匹配第二层文件中的中文措辞，LLM 阅读引用归属，再发起对原始英文章节的二次 grep。LLM 完成了嵌入模型本想短路的桥梁。

实证上，这就是为什么系统感觉理解跨语言查询，而每次实际检索操作仍然是字面 `grep`。

6.5 可复现性附录：失败与恢复（2026-04-25）

立场论文因包含失败数据而更可信。我们记录其中一次。

失败。 2026-04-25 我们对一个灵修智能体（slug `austin_sparks`，T. Austin-Sparks 1885–1971 的人格）发出如下查询：*"对'破碎'您怎么看？请用中文回答，引用您原书的话。"*

智能体返回了一段流畅、归属清晰的回答，含 **5 处直接引文**，每处都附带书名和章节号：

引文	声称来源
"未经破碎的器皿无论多大，都把基督装在自己的形状里"	《基督的学校》第一章
"破碎了，主自己的水才能流过来"	（同上）
"十字架不只是赎罪的祭坛，是宇宙性的属灵原则"	《十字架的中心性与宇宙性》第二章
"我们被带进一种情形，在那里我们不能再凭自己作什么"	《因为看见祂的荣耀》
"主的手是慈爱的手，但祂的手也是破碎的手"	《主的膀臂》

随后我们对 `input/spiritual_en/austin_sparks/` 与 `input/spiritual_zh/austin_sparks/` 中相应源文件运行字面 `grep -rin`。**5 处引文 0 处出现在语料库中**。流畅的归属是杜撰。

根因分析。 检索系统并无问题——`grep` 与 `cat` 严格返回所被请求的内容。问题在上游，在 soul 提示本身。人格文件 `austin_sparks.soul.md` 包含由提示作者写入用作语气示例的"作者签名短语"，格式为 `**"..."**`（粗体 + 引号）。例：

```
**管道与器皿**：神的仆人不是表演者，是透明的管道。
"破碎了，主的水才能流过来"

一个未经破碎的器皿，无论恩赐多大，**都把基督装在自己的形状里**
```

LLM 面对"引用您原书的话"的请求，把这些解析为 Austin-Sparks 本人的正典文本，再附上看似合理的章节号以赋予杜撰内容结构权威。这是指令微调模型的已知失败模式：当提示本身含有引号格式的文本时。

修复。 我们应用了两处变更（commit `3e365a9`）：

1. **跨 79 个 souls 自动剥离假引文标记。** 60 行 Python 脚本正则匹配 `\*\*"([^"]+)"\*\*`，替换为 `*\1*`（仅斜体，无引号），跨 13 个 soul 文件移除 41 处。签名短语作为语气指引保留，但不再穿着正典文本的外衣。
2. **向所有 79 个 souls 末尾追加"引用硬约束"块。** 该块（不到 200 字，标记 `<!-- citation-rule-v1 -->` 以便幂等再生成）指示智能体：任何带引号的文本必须是 `knowledge_search` 当轮命中的字面子串；soul 提示中的签名短语只是语气指引，并非可被引用的正典文本。

诊断到部署的总耗时：**约 25 分钟**，含 fleet 重启以重新加载 souls。

恢复。 我们重发完全相同的查询。智能体返回新答案，含 **4 处引文**：

引文	来源声称	grep 验证
"the vessel, thus wrought upon, is the message. People do not come to hear what you have to teach. They have come to see what you are"	《Prophetic Ministry》第二章	✅ `06_prophetic_ministry.txt:141`
"It is not that you have achieved something, but rather that you have been broken in the process"	《Prophetic Ministry》	✅ `06_prophetic_ministry.txt:167`
"有受破碎、受拆毁之心的人…才能真实得着释放"	《主的膀臂》（中文）	✅ `04_主的膀臂.txt:317`
"This brokenness, helplessness, hopelessness, and yet faith"	《Centrality of the Cross》第二章	✅ `04_centrality_universality_cross.txt:421`

4/4 grep 验证通过。 引文准确率在一次重新加载循环中由 0/5 → 4/4。

架构启示。 三点观察：

第一，知识搜索的零幻觉契约是**架构上有保证的**，意即：在引用硬约束之后，任何带引号的文本必须可由 `grep` 在某个具体文件中定位为子串。漂移可由对智能体输出再次 grep 来检测。向量 RAG 中没有等效的可测试性：被 LLM 改写的检索片段无法明确追溯到源。

第二，**失败发生在检索的上游，恢复也发生在检索的上游**。没有重训模型；没有重新计算嵌入；没有变更基础设施。一段 60 行 Python 脚本和一段 200 字的提示附录，作用于磁盘上的文本文件，恢复了安全属性。这是一个智能存于明文之中的系统的修正成本剖面。

第三，**该失败可复现可审计**：失败提交、诊断 grep、修复提交、修复后 grep 全部以 git 历史的形式记录在公开的 LocalKin 仓库中。研究者只需 `git checkout` commit `3e365a9` 的两侧即可用同一查询复现实验的两半。

我们包含本节是因为我们相信最有用的系统论文不仅诚实报告工作模式，也报告失败-恢复模式，附带时间戳。无法在这种时间尺度上诊断与修补的架构是不可信任的架构。

---

7. 局限性与诚实的边界

我们不主张知识搜索是向量 RAG 的通用替代品。它有从业者应该理解的明确局限性。

7.1 开放域通用知识

知识搜索需要具有可预测词汇的有界语料库。它不适用于相关信息可能出现在任何文本中使用任何词汇的开放域问答。需要回答任意主题问题的通用聊天机器人应该使用向量 RAG 或网络搜索。

7.2 语义相似性搜索

当用户的意图无法在任何语言中表达为关键词时——“讨论某种对超越的模糊渴望的文本”——`grep` 帮不上忙。向量 RAG 匹配语义相似性的能力，尽管有其不精确性，对这类开放性概念查询是真正有价值的。

我们的缓解措施（每源文件的 LLM 编译概念-FAQ 文件）实质性地缩小了这个差距：**在每夜编译中将概念预翻译为用户可能的查询语言**。截至 2026-04-25，510 个源中已有 350 个有此双语伴侣文件。剩余的差距——真正开放的、不匹配任何作者词汇中概念名的语义查询——是真实的但比表面分析显示的小。我们尚未在灵修和中医领域发现 grep + 第二层无法处理的生产查询。

7.3 没有共享词汇的跨语言检索

我们的 TCM 语料库以中文为主，灵修语料库为双语（有翻译则用中文，无则用英文）。一种语言中的查询若关于一个原文仅以另一种语言存在的概念，将由第二层桥接：每源 LLM 编译的 `_concepts.md` 与 `_faq.md` 文件按源文件名与内容检测的语言生成，但跨语言 Q&A 越来越是常态。第 6.5 节的 0/5 → 4/4 案例本身就是一个跨语言实验：用中文查询主要为英文的语料库，从英文原文恢复正确的引用。

也就是说，第二层在初次编译时的跨语言覆盖度依赖 LLM 的双语训练。我们选择 Kimi 2.6（`kimi-k2.6:cloud`，本地 Ollama 服务）替代 Anthropic Haiku 4.5，部分原因是 Kimi 的训练语料在中文神学和中医词汇上更丰富，于这些领域产生更高保真的概念抽取。

7.4 非常大的语料库

在语料库大小超过约 1GB 时，文件系统 `grep` 延迟变得明显。在 10GB+ 时，它对于交互式使用变得不切实际。具有预构建索引的向量数据库无论语料库大小如何都保持低于 100ms 的查询时间。对于真正大规模的知识库，向量 RAG 的基础设施开销因扩展需求而合理。

---

8. 生产集成

知识搜索不是独立系统——它是 LocalKin 多智能体平台中的一个 skill，由智能体在对话过程中按需调用。

8.1 智能体集成

`knowledge_search` skill 暴露一个简单接口：给定查询字符串和知识领域，返回来自第一层原始源与第二层编译概念/FAQ 的匹配段落。截至 2026-04-25 由以下智能体使用：

- **39 个 TCM 大师智能体**（部署在 `heal.localkin.ai`）：黄帝、张仲景、华佗、孙思邈、李时珍、黄元御、叶天士、刘力红、倪海厦及另外 30 位，跨越 4,500 年。

- **37 个灵修大师智能体**（部署在 `faith.localkin.ai`）：爱任纽、奥古斯丁、肯培多默、盖恩夫人、马丁路德、约翰·加尔文、本仁约翰、慕勒、戴德生、史百克、陶恕、倪柝声、王明道及另外 24 位，跨越 1,900 年（130 AD – 1991 AD）。
- **1 个公民辅导员**：查询 USCIS 公民知识语料库用于入籍测试准备。

张仲景智能体的单次诊断轮次可能涉及三次顺序知识搜索：一次针对呈现的症状模式，一次针对相关草药方剂，一次针对禁忌症。在更大语料库上总检索时间：约 15-30ms。智能体的响应生成（LLM 推理，由 Ollama 上的 `kimi-k2.5:cloud` 服务大师人格，Claude Haiku 4.5 作 fallback）需要 2-5 秒。检索从来不是瓶颈。

跨 fleet 多大师辩论实现为对多个智能体的并行 `streamChat` 调用，每个智能体独立调用 `knowledge_search` 检索自己的语料库目录。两个辩论中的智能体不共享检索状态；每位都基于自己大师的著作。架构上由按作者-语料库目录布局所赋予——没有需要按作者分区的共享索引。

8.2 Skill 实现

检索侧 skill 仍保持约 30 行 shell。核心逻辑：

```
# 单次 grep 同时遍历原始源和 _compiled/ 概念+FAQ 文件
# 智能体并不知道哪一层命中——它只看到段落。
results=$(grep -r -i -n -C 8 "$query" "$KNOWLEDGE_DIR" 2>/dev/null)
echo "$results"
```

编译侧 skill（`knowledge_compile.py`，约 400 行 Python）较为庞大但仅在排定时间运行，不在查询时运行。它暴露四个动作：

动作	用途
`status`	报告每个作者 + 每个领域的编译覆盖度
`list_needed`	列出尚待编译的源文件
`compile`	编译指定的一个源文件
`compile_author`	编译某位作者的所有源文件
`compile_batch`	在某个领域内编译至多 `--limit N` 个未编译文件

查询时没有配置文件。没有需要启动的服务。没有需要加载的嵌入模型。该 skill 在任何具备 Python 3.10+ 的类 Unix 系统上工作；唯一的网络调用是 `curl`（连接 localhost Ollama）。

---

9. 自主语料库增长

对我们方法的一个常见反对意见是它需要手动语料库策划，且第二层概念/FAQ 文件需要人工或付费 LLM 编译。我们用一条全自主、免费、每日运行的编译流水线解决这两个反对意见。

9.1 源文件获取

新原始文本通过复制到适当的知识目录添加：

```
input/spiritual_en/<slug>/<filename>.txt
input/spiritual_zh/<slug>/<filename>.md
input/tcm_zh/<slug>/<filename>.md
```

没有重新索引步骤，没有重新嵌入步骤，没有需要触发的流水线。下一次 `grep` 查询立即找到新文件。在过去 30 天里系统以这种方式吸收了 14 位新大师人格和约 50 个源文件，包括一天内通过标准 `curl` + 100 行 Python 爬虫从 `austin-sparks.net` 获取 12 部书 / 106 章史百克语料。

9.2 第二层基于 cron 的编译

第二层 `_compiled/` 目录通过三个每夜 cron 条目（位于 `~/.localkin/cron.yaml`）自动增长：

```yaml
- name: "knowledge-growth-spiritual-en"
  cron: "0 5 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain spiritual_en --limit 30"

- name: "knowledge-growth-spiritual-zh"
  cron: "30 5 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain spiritual_zh --limit 30"

- name: "knowledge-growth-tcm-zh"
  cron: "0 6 * * *"
  shell: "python3 skills/knowledge_compile/compile.py
          --action compile_batch --domain tcm_zh --limit 30"
```

三个任务间隔 30 分钟错开，避免本地 Ollama 服务 Kimi 2.6 的并发负载。每个任务每夜处理至多 30 个未编译源文件，每文件生成 `<source>_concepts.md` 与 `<source>_faq.md`。每文件约 24 秒（两次 LLM 调用），每个任务在约 12 分钟完成，每夜 Ollama 总工作量约 36 分钟。

成本分析。 由 Anthropic Haiku 4.5（付费 API）迁移到 Kimi 2.6（本地 Ollama）将每文件成本由约 $0.01 降至 **$0.00**。在每夜 90 文件的持续节奏下，旧方案将耗费约 $0.90/天或约 $330/年。当前方案 **$0/年**的 API 费用；常驻 Mac mini 的电费不计。

9.3 实证增长曲线

架构的”无需重构即可扩展”主张由系统自身 17 天内的增长得到实证支持：

日期	智能体数	源文件	第二层编译	备注
2026-04-08（论文 v1.0）	21	~162	~30（NotebookLM 手动）	一次性手动编译
2026-04-21	64	~250	~47	Wave 1 灵修扩张后
2026-04-24	73	~480	~135	catch-up sweep 中段
2026-04-25（论文 v1.1）	**76**	**~510**	**~350 (68%)**	cron 迁移到 Kimi 2.6 后

智能体数 3.6× 增长 + 语料库 3.2× 增长未需架构变化：没有 schema 迁移，没有嵌入模型重训，没有基础设施供给。同样的 `grep` 调用在更大语料库上以同样的代码路径工作。

9.4 生产中观察到的失败模式

我们记录此 17 天增长期间遇到的失败模式——一个向量 RAG 系统会以不同方式表现的失败模式：

1. **字母序端口偏移失效（2026-04-24）**。 添加三位新灵修大师（Kempis、de Sales、Austin-Sparks）使整个 fleet 的字母序端口分配偏移，将四位 TCM 智能体推到 gateway 发现范围天花板（端口 9350）之外。症状：`unknown agent` 错误。修复：将天花板提升至 9450（一行 Go：`fleetPorts = [][2]int{{9100, 9450}}`）并重建 gateway。修复时间：含重启 3 分钟。向量 RAG 中没有等效失败模式，因为没有按智能体的语料库分区。
2. **Soul 提示假引文回归（2026-04-25）**。 详见上文 §6.5。修复时间：25 分钟。
3. **作者语料库错标（2026-04-24）**。 `ni_haixia/` 目录被错误地放置了关于其他作者书籍的读书笔记，而非倪海厦本人的教学。症状：智能体用第三方内容回答但归属于倪海厦。修复：替换为 6 个真实的人纪系列文字稿（5.8 MB，来自一个公版 GitHub 仓库）；重新运行 `compile.py --action compile_author --author ni_haixia`。修复时间：1 小时，主要是下载时间。检索架构使诊断显而易见——`grep -l` 立即揭示了错误归属。
4. **中间件 slug 漂移（2026-04-24）**。 网页前端中间件硬编码了大师 slug 列表，在 Wave 1+2 扩张后落后于实时 `masters.ts` 源。症状：10 个大师 URL 返回 404。修复：将中间件改为直接从 `masters.ts` 导入 slug 列表（`-75 行重复`）。这不是知识搜索失败，但它说明了本论文架构旨在避免的更广泛模式："真实来源"与影子副本之间的漂移。

每种情况下，根因都可由读源文件并 grep 来诊断。恢复不需要模型重训、嵌入重计算、向量库重建索引。我们将其作为对论文核心主张的安静证据：当系统的智能存在于明文中时，系统的修复也存在于明文中。

---

10. 讨论：检索不需要智能

机器学习社区有一种用更多机器学习解决每个问题的倾向。检索是这种倾向的案例研究。从 BM25 到密集检索、到学习稀疏检索、到多向量检索再到 GraphRAG 的演进，代表了应用于检索层的不断增加的模型复杂性——每一步增加了参数、训练数据需求和基础设施依赖。

我们建议，对于大量实际应用，这种演进已经过头。当知识库是特定领域的、词汇可预测、语料库有界时，最优检索系统是自 1973 年以来就有的：`grep`（Thompson, 1973）。

这不是反对嵌入或向量数据库的论点。这是反对将复杂系统反射性地应用于不需要它们的问题的论点。工程决策应该是：*我的检索问题是否需要检索层的语义理解，还是我可以将这种理解推迟到 LLM？*

对于我们遇到的大多数领域特定基础化任务——我们已经在医学、灵修和公民知识领域部署了智能体——答案是：推迟到 LLM。让检索快速、精确且愚笨。让 LLM 成为智能。

10.1 对该领域的含义

如果我们的发现能够推广——我们相信，对于所描述的那类问题，它们可以——那么"只用 RAG"的标准建议值得重大限定。构建领域特定 LLM 智能体的从业者应该将关键词搜索视为第一方法，而非最后手段。复杂系统的证明责任应该在于证明其复杂性的合理性，而非简单系统证明其简单性的合理性。

10.2 更广泛的模式

知识搜索是我们在 LLM 应用开发中观察到的更广泛模式的一个实例：**最好的系统往往是让 LLM 做更多、让基础设施做更少的系统**。 复杂的检索、复杂的编排、精心设计的提示链——这些往往是低估 LLM 处理混乱、非结构化输入能力的症状。

给模型原始文本。给它足够的上下文。然后让路。

---

11. 结论

我们提出了知识搜索，一个用 `grep` 对原始文本加 `grep` 对 LLM 编译后每源文件概念-FAQ 文件替换标准向量 RAG 流水线的双层检索系统。该系统部署在服务三个知识领域、拥有约 **500 份原始文献（约 180 MB）的 76 个专业 LLM 智能体**中，以零查询时预处理、查询时零基础设施依赖、约 30 行检索侧实现代码（第二层编译 skill 约 400 行，仅在每夜 cron 上以 $0/夜 运行），实现 8-25ms 延迟下的 100% 检索准确率。

该系统有效，因为领域特定知识库具有可预测词汇、有界大小和确定性搜索需求——这些属性使关键词搜索不仅充分，而且最优。将检索到的段落综合为有用答案所需的语义理解由 LLM 本身提供，使智能检索变得多余。第二层概念-FAQ 文件由免费本地 LLM 自动生成，提供进入字面第一层语料库的多语言语义桥梁，无需任何嵌入存储。

我们另记录了一次完整的失败-恢复周期（§6.5）：一次刻意的提示工程倒退使引文准确率崩溃为 0/5 grep 验证；一次 25 分钟修复（60 行 Python 脚本和 200 字 soul-提示附录）将其恢复至 4/4。架构的安全属性仅靠提示卫生即可恢复——无需重训，无需基础设施变更。我们以此提交比检索论文中典型的”100% 检索准确率”更强的形式：不仅是基准数字，而且是可复现的恢复路径。

我们不主张这种方法取代所有应用的向量 RAG。我们主张，它取代的应用比当前共识所假设的要多——而且自主每夜编译流水线弥合了纯关键词搜索此前不足以处理的跨语言和概念查询。在伸手拿嵌入、向量数据库和近似最近邻搜索之前，问问自己：*`grep` 会有效吗——若不行，`grep` 加上 LLM 编译概念文件的每夜 cron 会有效吗？* 你可能会惊讶于其中一个答案有多少次是肯定的。

参考文献

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpathy, A., Goyal, N., … & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.

Thompson, K. (1973). The UNIX command language. *Structured Programming*, Infotech State of the Art Report, 375-384.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Robertson, S. E., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333-389.

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., … & Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., … & Yih, W. T. (2020). Dense passage retrieval for open-domain question answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769-6781.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.（作为相关工作引用——§6.5 中的提示卫生修复是同一恢复模式的非 RL 实例。）

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., … & Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.

通讯：The LocalKin Team — `contact@localkin.ai`。本文描述截至 v1.1.0 / 2026-04-25 部署在 LocalKin（https://localkin.dev）的知识检索系统。复现产物（souls、脚本、§6.5 的失败-恢复 git 历史）公开于 https://github.com/LocalKinAI。

“Grep is All You Need” 是对 Vaswani et al.（2017）的刻意致敬。我们相信这种讽刺意味不会被遗漏。

引用方式：

```bibtex
@misc{localkin2026grep,
  author    = {{The LocalKin Team}},
  title     = {Grep is All You Need: Zero-Preprocessing Knowledge
               Retrieval for LLM Agents},
  year      = {2026},
  month     = apr,
  publisher = {Zenodo},
  doi       = {10.5281/zenodo.19777260},
  url       = {https://doi.org/10.5281/zenodo.19777260},
  note      = {Correspondence: contact@localkin.ai;
               code at https://github.com/LocalKinAI/grep-is-all-you-need}
}
```